# ■ Phase 4: Project Design

## Solution Architecture

| | |
|---|---|
| **Date:** | July 8, 2026 |
| **Team ID:** | SWTID-2026-4823 |
| **Project Name:** | BOOK STORE (BookNest) |

## System Architecture

BookNest uses a stateless Express API backend and React frontend client communicating over HTTP REST protocols.

### 1. Data Layer: Database Schema Models

- **UserSchema:** Stores name, email, passwordHash, role (user/seller/admin), status (approved/pending/suspended), and wishlist (array of book IDs).
- **BookSchema:** Stores title, author, genre, price, stock, description, rating, featured, badge, cover colors object, tags, itemImage, and seller ID/name references.
- **OrderSchema:** Stores buyer details, expected delivery, shipping address sub-schema, items array (bookId, title, price, quantity, sellerId), subtotal, shippingFee, and total amount.

### 2. Routing & Controllers Layer: REST Endpoints

- Auth routes: /api/auth/register (POST), /api/auth/login (POST), /api/auth/me (GET)
- Catalog routes: /api/books (GET/POST), /api/books/:id (GET/PUT/DELETE)
- Wishlist routes: /api/wishlist (GET), /api/wishlist/:bookId (POST/DELETE)
- Orders routes: /api/orders (POST), /api/orders/me (GET), /api/orders/inbox (GET), /api/orders/:id/status (PATCH)
- Admin dashboard: /api/admin/dashboard (GET), /api/admin/users/:id/status (PATCH)

### 3. Checkout Request Data Flow

When a customer checks out their cart:
1. Client UI submits cart items and shipping details via POST request to `/api/orders` with JWT headers.
2. Backend `authMiddleware` validates JWT token using `jwt.verify` and decodes the buyer email/ID.
3. The database updates book stock counts using Mongoose `$inc: { stock: -qty }` queries for each purchased book ID.
4. A new document is written to the Orders collection, and the customer's cart is set to empty in the database.
5. Express returns order details in a JSON payload; React displays the order summary screen.